IA#2 — Zero-downtime migration · rubric

CSC13114 · individual assignment · 3% of the course · 100 points
Change menu_items.price (thousands of dong, a float) into menu_items.price_vnd (dong, an integer) in the menu service of smart-restaurant, while two instances keep serving traffic — then drop price. No failed request at any point, and a way back at every step until the one you name as the point of no return. Start from the starter repository linked in this assignment. Its README.md explains how to run two versions at once and what the traffic script counts.

What you hand in

One zip of your repository, **including .git**, so that your tags can be checked out. It must contain:

File	What it is
tags v1, v2, …	one tag per code version you deployed, v1 = the starter
migrations/NNN_*.up.sql + .down.sql	one pair per schema step
your backfill script	under scripts/
MIGRATION.md	the plan, as a table of steps (see criterion 1)
traffic-log.txt	**one** continuous run of npm run traffic, from v1 to after the last migration
check-log.txt	the output of npm run check after every step, each run labelled with its step
AI-LOG.md	as for every assignment
SELF_ASSESSMENT_REPORT.md	see *How to submit*

What is scored

#	Criterion	Points
1	The plan	20
2	Expand and dual write	20
3	Backfill	15
4	Zero failed requests	20
5	The way back	15
6	Contract	10

1. The plan — 20 points

MIGRATION.md has one row per step. Each row gives: the schema after the step, the code version every instance runs, the exact command that undoes the step, and which earlier versions can still run safely against this schema.

Points	What it looks like
17–20	Every step has all four columns. The point of no return is named, with the reason it cannot be undone. The table matches the tags and migrations in the repository.
11–16	All steps are there, but the undo command or the compatibility column is missing or vague for some of them.
4–10	A list of steps in prose, with no way back described.
0–3	No plan, or a plan that does not match the repository.

2. Expand and dual write — 20 points

Points	What it looks like
17–20	The new column is added in its own migration and v1 keeps working on it. One version writes **both** columns on **every** write path — creating an item and re-pricing one. The API offers priceVnd and still returns price. Every migration has a .down.sql, and migrate:down was run once for each — on a separate database file is fine — with the output in check-log.txt.
11–16	Expand and dual write are there, but one write path writes only one column, or one migration has no down.
4–10	The schema change and the code switch happen in the same step.
0–3	A rename or a type change in place.

3. Backfill — 15 points

Points	What it looks like
13–15	Runs in small batches, each its own short transaction, while traffic runs. Can be stopped and

Points	What it looks like
	started again with the same result. Runs only when no version that writes price alone is still serving. check-log.txt shows CONSISTENT right after it.
8–12	Works, but in one long transaction, or the log does not show CONSISTENT after it.
3–7	Run before the old version was gone, or not re-runnable.
0–2	No backfill, or the data was converted by hand.

4. Zero failed requests — 20 points

Points	What it looks like
17–20	One continuous traffic-log.txt covers every deploy and every migration, and ends with RESULT: zero failed requests. The rolling restarts are visible as down / back lines for one instance at a time. NEW_CLIENTS was above 0 for at least the part after the API offered priceVnd.
11–16	Zero failures, but the log is split into several runs, or NEW_CLIENTS was never turned on.
4–10	A few failures, explained in MIGRATION.md with what caused them.
0–3	Failures not explained, or no log.

5. The way back — 15 points

Points	What it looks like
13–15	Under traffic, the version that **reads** price_vnd was rolled back to the version before it, then forward again — zero failures in the log for that stretch, and CONSISTENT before and after. MIGRATION.md says why that rollback is safe and which later rollback would not be.
8–12	A rollback was done, but not under traffic, or not from the version that reads the new column.
3–7	Rollback described, never performed.
0–2	No rollback.

6. Contract — 10 points

Points	What it looks like
9–10	price is dropped only after a version that neither reads nor writes it is on every instance. After the drop, old clients still get price in every response, and the log stays clean.
5–8	Dropped at the right time, but one endpoint stops returning price.
1–4	Dropped while a version that uses it was still serving.
0	Not dropped.

How to submit

One zip file, named after your student ID **and the total you give yourself**: <StudentID>_<total>.zip, for example 23120001_82.zip.
The zip must contain **SELF_ASSESSMENT_REPORT.md**: one row per criterion in this rubric, the marks you claim, and evidence pointing at a file, a tag, a line range of traffic-log.txt, or a migration — plus a short *what I did not manage*. The total in that table is the number in the file name. Template on Classroom.
A wrong file name costs no marks but delays your result — the grading script matches submissions to the class list by that name.

Honesty adjustment

Your self-assessment is compared with the mark you actually earn. The gap is your total − the mark, on the same 100-point scale.

Gap	Adjustment
within ±10	none — this is normal calibration
+11 to +20	−3
+21 to +30	−6
more than +30	−10
−21 or worse	−3 — read the rubric before you score yourself down
no SELF_ASSESSMENT_REPORT.md	−10, and the file-name total is ignored

A criterion you claim with no evidence line counts as claimed-and-not-done for this comparison. The adjustment never takes a submission below 0.

Notes

- The logs are evidence, not the grade. We may check out your tags and replay the sequence; a log that the repository cannot reproduce scores 0 on criteria 4–5.
- traffic-log.txt only sees rows that were read. A row that is wrong but was never read shows up only in npm run check — that is why both logs are required.
- SQLite is used so that the starter needs no installation. In PostgreSQL the same plan needs extra care with locks — that is in the session 4 slides, and it is fair game for the exam.
- You may use an assistant for any of it. Say so in AI-LOG.md. Remember the session 3 tiers: the assistant may write a migration file; you run it.